

Este es el plan maestro estructurado para un proyecto de **18 semanas (4 meses y medio)**, dividido en **6 Sprints de 3 semanas cada uno**.

El cronograma integra los requisitos técnicos específicos de cada grupo con la ruta de aprendizaje obligatoria (**HF Agents, MCP, A2A y CopilotKit**).

Cronograma General de Sprints (18 Semanas)

Sprint	Enfoque Principal	Hito de Formación (Training)
S1	Ingestión y Chunking Multimodal	HF Agents Course (Unit 0) & MCP Basics
S2	Bases de Datos y Ontologías	MCP Server Implementation (Local/Stdio)
S3	Lógica de Agentes (ADK/LangGraph)	A2A Protocol (Agent-to-Agent)
S4	Sistemas Multivector y MUVERA	Research: Métricas de Similitud Avanzada
S5	Interfaz Humano-en-el-bucle	CopilotKit (State Management & Canvas)
S6	Evaluación RAGAS y Optimización	LangSmith Tracing & Observabilidad

Tareas Detalladas por Grupo

Grupo 1: Neo4j, ImageBIND (CLIP), Langgraph(ADK)

Grupo 2: Qdrant, CLIP (ImageBIND), Langgraph(ADK)

Grupo 3: Qdrant, ImageBIND(CLIP), Langgraph(ADK)

Grupo 4: Qdrant, ColPALI, MUVERA, Langgraph(ADK)

Grupo 1: Nicolas Mazzeo - Tom Pinagote

Grupo 2: Mateo Gonzalez Pautaso - Alen Calandria

Grupo 3: Andrea Figueroa Rodriguez - Tiago Calderon

Grupo 4: Luis Fernando Peña Ramirez - Nicolás Romano.

Métricas y evaluación: Raquel Ana Davila - Leticia Figueroa Rodriguez - Gabriel Narvaez Yaguana

Sprint 1: Chunking e Ingestión

- Extraer texto, tablas e imágenes del manual de histología. Con embeddings multimodales CLIP o ImageBIND.
- **Investigación:** Determinar el tamaño de chunk óptimo para que CLIP o ImageBIND mantenga el contexto semántico de una microfotografía junto a su descripción.
- Extracción inicial de la ontología (entidades jerárquicas: Órgano -> Tejido -> Célula).

● Sprint 2: Persistencia

- G1: Modelar la ontología en Neo4j (Nodos y Relaciones) y añadir vectores CLIP o ImageBIND a los nodos.
- Investigar algoritmos de RAG como ANN, HNSWy posibilidades de mejora.
- Investigar Embeddings para multimodalidad
- G2/G3: Configurar colecciones en Qdrant con vectores densos (Coseno).
- Probar cuál de las dos bases datos da mejores resultados en la consulta.

● Sprint 3: Orquestación Agéntica

- Configurar el grafo en **LangGraph** o **ADK**.
- Implementar la búsqueda híbrida: El agente usa la ontología para filtrar el espacio de búsqueda antes de aplicar la similitud de coseno.

● Sprint 4: Comunicación A2A

- Implementar protocolo **A2A** para que el agente de búsqueda pueda interactuar con un agente de "explicación médica".
- Ver la importancia de MCP en los agentes.
- Validar la recuperación de "elementos conocidos" (ej: lámina de tejido linfoide).

● Sprint 5: Integración CopilotKit

- Desarrollar la UI con **CopilotKit**.
 - Implementar `useCoAgent` para mostrar en el Canvas la imagen del manual que el agente está analizando.
 - **Sprint 6: RAGAS y LangSmith**
 - Medir *Faithfulness* y *Context Recall* con RAGAS.
 - Ajustar prompts basándose en las trazas de LangSmith.
-

Grupo 4 (Híbrido Avanzado: ColPali, ColBERT, MUVERA)

Diferenciador: Estado del arte en recuperación multivectorial.

- **Sprint 1: Chunking de Interacción Tardía**
 - Investigar chunking para **ColBERT** (token-level) y **ColPali** (basado en parches de imagen/página completa).
 - Setup de Qdrant con soporte para **Named Vectors** (múltiples vectores por punto).
- **Sprint 2: Servidores MCP Multimodales**
 - Implementar servidores MCP que manejen no solo texto, sino el pasaje de tensores/parches de imágenes.
 - Indexación masiva de páginas del manual usando ColPali.
- **Sprint 3: MUVERA y Orquestación**
 - Configurar LangGraph para manejar la complejidad de múltiples modelos (CLIP + ColBERT + ColPali).
 - Implementar **MUVERA** para la selección eficiente de candidatos multivectoriales antes del re-ranking.
- **Sprint 4: Métricas Multivector (Investigación)**
 - **Tarea Crítica:** Implementar **MaxSim** (Maximum Similarity) para la interacción tardía.
 - Investigar e implementar **RRF (Reciprocal Rank Fusion)** para combinar los scores de CLIP y ColPali.
- **Sprint 5: UI Compleja con CopilotKit**
 - Usar CopilotKit para permitir al usuario "señalar" partes de una imagen de histología (Human-in-the-loop).
 - Sincronizar el estado del agente multivectorial con la respuesta visual en el Canvas.
- **Sprint 6: Evaluación y Comparativa**

- Comparar formalmente (con RAGAS) si el sistema multivectorial del G4 supera a los sistemas densos de los G1, G2 y G3 en la identificación de estructuras celulares pequeñas.
-

Resumen de Métricas de Similitud a Entregar

1. **Vectores Densos (G1, G2, G3):**
 - **Similitud de Coseno:** Obligatoria para embeddings CLIP normalizados.
2. **Sistemas Multivector (G4):**
 - **MaxSim (Late Interaction):** Para medir la relevancia entre los términos de la consulta y los parches de imagen (ColPali).
 - **Borda Count o RRF:** Para la fusión de rankings híbridos.
 - **Distancia de Chamfer (Opcional):** Investigar su viabilidad para comparar conjuntos de vectores de imagen vs texto.

Entregables de Formación por Sprint

- **Final del Sprint 1:** Certificado/Prueba de Hugging Face Agents Unit 0.
- **Final del Sprint 2:** Servidor MCP funcional capaz de leer el manual de histología.
- **Final del Sprint 3:** Demostración de un agente llamando a otro (A2A) para resolver una duda del manual.
- **Final del Sprint 5:** Aplicación web funcional con CopilotKit donde el usuario puede "co-editar" la investigación con el agente.

NotebookLM

<https://notebooklm.google.com/notebook/d532c7db-737c-4916-9f22-5ba6c2f5eed0?authuser=1>

CURSOS

<https://huggingface.co/learn/agents-course/unit0/introduction>

<https://huggingface.co/blog/tsadoq/agent2agent-and-mcp-tutorial#multiple-local-mcp-servers-where-stdio-is-not-enough>

RAG y EBEDDINGS

<https://learn.deeplearning.ai/courses/retrieval-optimization-from-tokenization-to-vector-quantization/lesson/lpcaz/introduction>

<https://learn.deeplearning.ai/courses/multi-vector-image-retrieval/lesson/opbx8g/introduction>

<https://learn.deeplearning.ai/courses/agentic-knowledge-graph-construction/lesson/hhqjv/introduction>

<https://learn.deeplearning.ai/courses/knowledge-graphs-rag/lesson/mltr8/introduction>

A2A tutorial

<https://learn.deeplearning.ai/courses/a2a-the-agent2agent-protocol/lesson/vtf72ap4/introduction>

<https://a2a-protocol.org/latest/tutorials/>

CopilotKIT

<https://docs.copilotkit.ai/>

A2UI

<https://docs.copilotkit.ai/a2a/generative-ui/declarative-a2ui>

Recursos de visualización

<https://playground.tensorflow.org/#activation=tanh&batchSize=10&dataset=circle®Dataset=reg-plane&learningRate=0.03®ularizationRate=0&noise=0&networkShape=4,2&seed=0.87303&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false>

<https://projector.tensorflow.org/>

Links: Código + Teoría — RAG Evaluation, ColPALI/MUVERA & RAG Agéntico



RAGAS — Documentación Oficial y Código

Link	Qué encontrarás
docs.ragas.io — Quickstart	Setup inicial, métricas básicas, integración con OpenAI/Claude/Gemini
docs.ragas.io — Evaluar RAG	Tutorial completo de evaluación de un sistema RAG desde cero
docs.ragas.io — Mejorar un RAG App	Ejemplo end-to-end: detectar fallas de retrieval → pasar a RAG agéntico
docs.ragas.io — Evaluación de Agentes	Métricas para agentes: Tool Call Accuracy, Agent Goal Accuracy, Topic Adherence
docs.ragas.io — API Reference <code>evaluate()</code>	Referencia del método principal con ejemplos de código
GitHub RAGAS	Repositorio oficial, issues, notebooks de ejemplo

¿Son las mismas metricas para ColPALI-MUVERA que con los otros RAGs?

Buena pregunta. La respuesta corta es: **no exactamente** — hay métricas compartidas, pero ColPALI+MUVERA introduce métricas únicas que no existen en RAG clásico. Te lo desgloso:



Métricas que Sí comparten todos los RAGs

Estas son universales porque evalúan la *calidad de generación*, independiente del retriever:

Métrica	RAG Clásico	RAG CoIPALI+MUVERA
Faithfulness	✓	✓
Answer Relevancy	✓	✓
Context Precision	✓	✓
Context Recall	✓	✓
Factual Correctness	✓	✓
nDCG@K, Recall@K, MRR	✓	✓
Latencia end-to-end	✓	✓
Cost per Query	✓	✓

Métricas ÚNICAS de CoIPALI+MUVERA (no existen en RAG clásico)

Aquí está la diferencia real:

1. Por ser multimodal (CoIPALI)

RAG clásico trabaja con texto puro. CoIPALI trabaja con **páginas como imágenes**, lo que genera métricas que no tienen equivalente:

Métrica	Por qué no existe en RAG clásico
nDCG@5 sobre ViDoRe	ViDoRe es un benchmark visual. RAG clásico usa BEIR (solo texto)
Multimodal Faithfulness	RAG clásico no tiene contexto visual que verificar
Multimodal Context Relevance	Evalúa si una <i>imagen de página</i> es relevante, no un chunk de texto
Patches per Page	Los patches de ViT no existen en retrievers de texto
Similarity Maps (visualización)	Mapas de atención por zona de imagen — imposible en texto
Indexing Throughput (pages/s)	En RAG clásico se mide en tokens/s o chunks/s, no páginas visuales

2. Por ser multi-vector con FDE (MUVERA)

RAG clásico usa vectores únicos (dense retrieval tipo OpenAI embeddings o BM25). MUVERA usa multi-vector late interaction, lo que introduce:

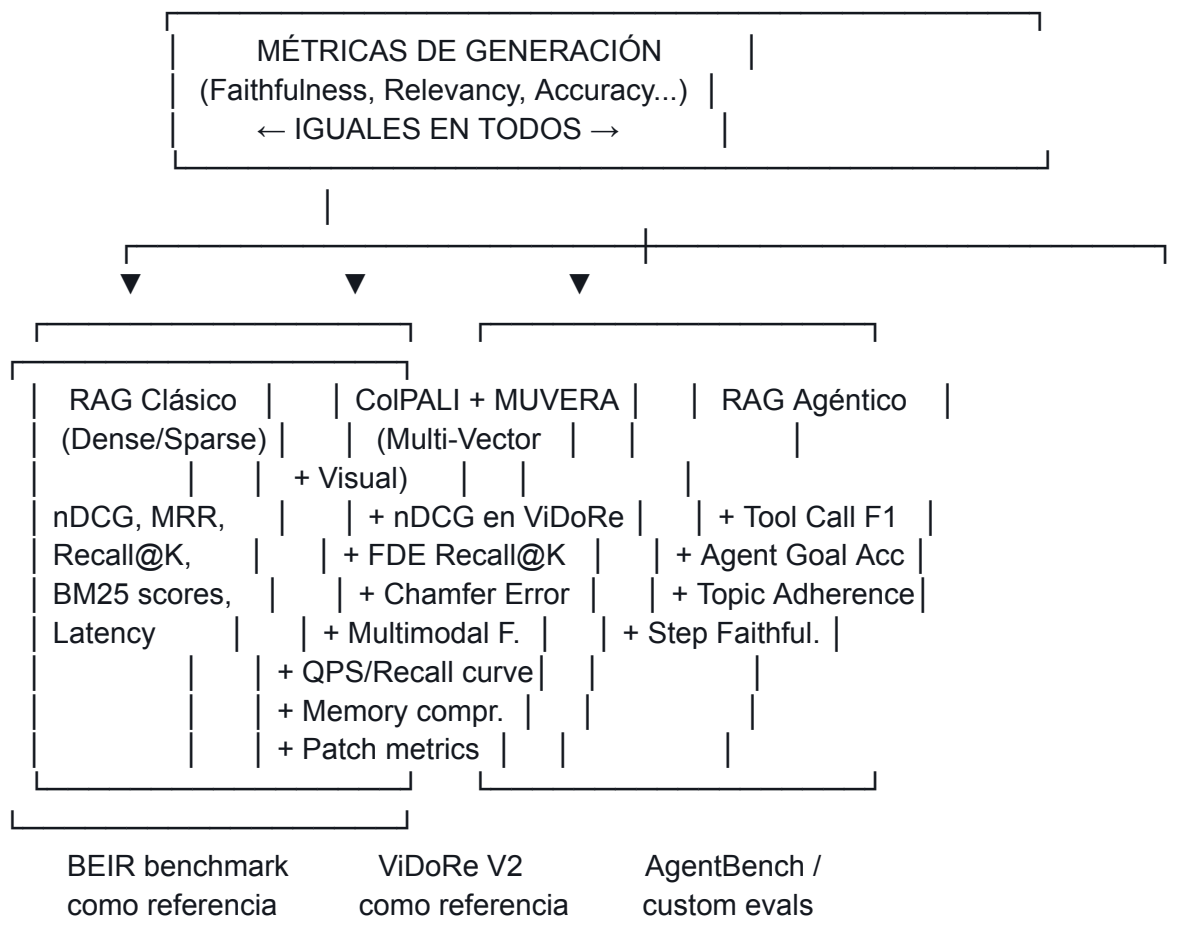
Métrica	Por qué no existe en RAG clásico
FDE Recall@K vs. exact MaxSim	Mide pérdida de la <i>aproximación</i> FDE respecto al MaxSim exacto — concepto inexistente en single-vector
Chamfer Similarity Error (ϵ)	El cálculo de similitud Chamfer es exclusivo de multi-vector
Nodos HNSW por documento	En RAG clásico 1 doc = 1 nodo. En MUVERA es el punto clave de la compresión
Compresión FDE (%)	La reducción de 1.9B $\rightarrow$ 304M floats no aplica a embeddings únicos
QPS vs. Recall@K curve	En RAG clásico el recall no varía con QPS de la misma manera; en MUVERA es el trade-off central
k_sim / d_proj / r_reps sensitivity	Son hiperparámetros exclusivos de MUVERA sin análogo en RAG clásico

3. Por ser RAG agéntico (si aplica)

Si el pipeline incluye un agente encima de ColPALI+MUVERA, se suman métricas que tampoco existen en RAG clásico estático:

Métrica	Por qué no existe en RAG clásico
Tool Call Accuracy / F1	RAG clásico no tiene tool calls
Agent Goal Accuracy	RAG clásico no tiene planificación de objetivos
Topic Adherence	RAG clásico no tiene conversación multi-turno con scope
Step-level Faithfulness	Verifica cada paso del razonamiento, no solo la respuesta final

Mapa visual de las diferencias



🎯 La diferencia más importante en la práctica

En RAG clásico, el retriever es una **caja negra simple**: metes texto, salen chunks. Las métricas de retrieval son relativamente sencillas (nDCG, Recall).

En ColPALI+MUVERA, el retriever tiene **dos capas de aproximación** que cada una introduce su propia pérdida de calidad:

Query → [ColPALI: texto → patches multi-vector] ← pérdida 1: calidad visual
 → [MUVERA: multi-vector → FDE single-vector] ← pérdida 2: aproximación FDE
 → [HNSW: exact → approximate search] ← pérdida 3: ANN approximation

Por eso necesitas métricas en **cada capa**, cosa que en RAG clásico con un solo vector denso simplemente no aplica.